JPA Relationships - Complete What & Why Notes

@ManyToOne

What?
Many child entities belong to one parent entity.

Why do we need it?
A Book always belongs to one Author, while an Author can write many Books. Instead of storing the whole Author object inside the books table, Hibernate stores only the author's primary key as a foreign key (author_id).

Example
Book -> Author

Interview Line
Used when many records reference one parent. Foreign key is stored in the child table.

@OneToMany

What?
One parent entity contains multiple child entities.

Why do we need it?
It is the reverse view of ManyToOne. It allows an Author object to access all of its Books. The foreign key is NOT stored here; it is already stored in the Book table.

Example
Author -> List

Interview Line
Normally the non-owning side. Uses mappedBy.

@OneToOne

What?
One entity maps to exactly one other entity.

Why do we need it?
A Book has one BookDetail (ISBN, pages, language, publisher). BookDetail cannot meaningfully exist for multiple books.

Example
Book -> BookDetail

Interview Line
Usually used for additional information that belongs to only one record.

@ManyToMany

What?
Many entities relate to many entities.

Why do we need it?
A Book can belong to many Categories and a Category can contain many Books. One foreign key cannot represent this relationship, so Hibernate creates a bridge table.

Example
Book <-> Category

Interview Line
Hibernate automatically creates a join table.

@JoinColumn

What?
Defines the foreign key column.

Why?
It tells Hibernate exactly where to store the foreign key. Example: author_id inside books.

@JoinTable

What?
Defines the bridge table for a Many-to-Many relationship.

Why?
Neither table can store multiple foreign keys in one column, so Hibernate creates book_category(book_id, category_id).

mappedBy

What?
Marks the non-owning side of a relationship.

Why?
Without mappedBy, Hibernate thinks both entities own the relationship and may create duplicate mappings. The owner side contains @JoinColumn.

Cascade

What?
Propagates operations like save, update and delete from parent to child.

Why?
If the child's lifecycle depends on the parent, Hibernate can automatically perform the same operation on the child.

Memory Rule: Ask: *Can the child survive without the parent?*

FetchType

LAZY - Load related data only when it is accessed.

EAGER - Load related data immediately with the parent.

Why?
Use LAZY for large collections (OneToMany, ManyToMany). Use EAGER for small related objects that are frequently required (ManyToOne, OneToOne).

Orphan Removal

What?
Deletes a child entity when it is removed from the parent's relationship.

Why?
Prevents orphan records from remaining in the database after the parent no longer references them.

Difference from Cascade REMOVE:
Cascade REMOVE = delete parent -> delete child.
orphanRemoval = parent stays, relationship removed -> delete child.

Golden Rules

1. One Entity = One Table.
2. Primitive field = Database Column.
3. Entity field = Foreign Key.
4. ManyToMany = Join Table.
5. Owner side contains @JoinColumn.

6. Non-owner side uses mappedBy.
7. Choose relationships based on business requirements.
8. Choose Cascade based on lifecycle dependency.
9. Choose FetchType based on how often related data is needed.